

- Now I have all the struct definitions. Let me compile the full analysis.

0x27 服务 — 完整配置链路与调用关系详解

一、配置文件总览

DaVinci Configurator 生成两个文件，所有配置在 Flash 中以 `CONST` 数组存放：

文件	内容
Dcm_Lcfg.h	结构体类型定义 + extern 声明
Dcm_Lcfg.c	静态配置数组实例

二、0x27 服务涉及的结构体（一级一级串联）

第一层：服务入口 — `Dcm_CfgDiagServiceInfo[]`

```
// Dcm_Lcfg.h 行745-760
struct DCM_CFGDIAGSERVICEINFOTYPE_TAG {
    Dcm_DiagSvcProcessorFuncType ProcFunc; // 服务处理器，如 Dcm_Service27Processor
    DcmCfg_DiagServicePropertiesType Props; // 属性位图(bit0:需L1解锁, bit1:需L2解锁)
    uint8 MinLength; // 最小请求长度(不含SID)
    uint8 ConfFuncRef; // 后处理器索引
    uint8 FastConfFuncRef;
    uint8 UpdateFuncRef;
    uint8 CancelFuncRef; // 取消处理器索引
};
```

C801 变更项：

```
- Props = 0x03u // bit0+bit1 set → 需L1+L2解锁 → 需L3安全等级
+ Props = 0x02u // bit1 set → 需L2解锁 → 需L2安全等级
```

第二层：服务 -- 前置条件 — `Dcm_CfgDiagSvcIdExecPrecondTable[]`

这是个 `uint8` 数组，每个 SID 对应一个索引，指向 `Dcm_CfgStatePreconditions[]` 数组的条目：

```
// Dcm_Lcfg.c: 0x27的ExecCondRef = 3u
// 指向 Dcm_CfgStatePreconditions[3]
```

C801 变更项： `0u` → `3u`

第三层：前置条件 — `Dcm_CfgStatePreconditions[]` + `Dcm_CfgModeMgrRules[]`

```
// Dcm_Lcfg.h 行352-360
struct DCM_CFGSTATEPRECONDITIONINFOTYPE_TAG {
    Dcm_CfgModeMgrRuleRefMemType ModeRuleRef; // 指向 Dcm_CfgModeMgrRules[] 条目
    Dcm_CfgStateDiagStateInfoType States; // 诊断状态要求(位图: Session + Security)
};
```

```
// Dcm_CfgStateDiagStateInfoType 行340-349
struct DCM_CFGSTATEPRECONDITIONTYPE_TAG {
    Dcm_CfgStateGroupMemType Session; // 要求的会话状态(位图)
    Dcm_CfgStateGroupMemType Security; // 要求的安全访问状态(位图)
};
```

```
// Mode Rule 函数指针类型 行89
typedef P2FUNC(boolean, DCM_CODE, Dcm_ModeRuleFuncType)(Dcm_Ptr2tLocalU8Type nrc);
```

第四层：安全状态 — `Dcm_CfgStateSecurityInfo[]`

```
// Dcm_Lcfg.h 行372-386
struct DCM_CFGSTATESECURITYINFOTYPE_TAG {
    Dcm_TmrTimerCntrMemType DelayTimeInvKey; // 输错key的惩罚延时
    Dcm_TmrTimerCntrMemType DelayTimeOnBoot; // 上电延时
    Dcm_TmrTimerCntrMemType ReloadTime; // AttCnt读取失败时的延时
    boolean ReloadOnZeroDelay;
    uint8 NumAttempts; // 最大尝试次数
    uint8 Value; // 安全等级ID (= (SF+1)/2, L1=1, L3=2)
    Rte_ModeType_DcmSecurityAccess Mode; // 安全访问RTE模式
};
```

C801 当前值（行440-443）：

```
{ 1000u, 0u, FALSE, 3u, 0x01u }
// DelayTimeInvKey=1000ms, DelayTimeOnBoot=0, NumAttempts=3次, Value=0x01(L1)
```

第五层：27子功能 — `Dcm_CfgSvc27SubFuncInfo[]` + `Dcm_CfgSvc27SecLevelInfo[]`

```
// Dcm_Lcfg.h 行1133-1138 — 子功能配置
struct DCM_CFGSVC27SUBFUNCINFOTYPE_TAG {
    Dcm_CfgNetBufferSizeMemType ReqLength; // 请求数据长度(含SF字节)
};
```

```
// Dcm_Lcfg.h 行1141-1152 — 安全等级-OEM回调映射
struct DCM_CFGSVC27SECLEVELINFOTYPE_TAG {
    Dcm_Svc27GetSeedFuncType GetSeedFunc; // → SecurityAccess_Unlocked_0x01_GetSeed
    Dcm_Svc27CompareKeyFuncType CompareKeyFunc; // → SecurityAccess_Unlocked_0x01_CompareKey
```

```
Dcm_Svc27CompareKeyFuncType CompareKeyFunc; // SecurityAccess_Unlocked_0x01_CompareKey
Dcm_Svc27GetAttemptCntFuncType GetAttemptCntFunc; // GetAttemptCounter
Dcm_Svc27SetAttemptCntFuncType SetAttemptCntFunc; // SetAttemptCounter
Dcm_CfgNetBufferSizeMemType SeedResLength; // seed响应长度
Dcm_Svc27OpClassType GetSeedFuncClass; // 函数原型描述符
};
```

C801 变更项:

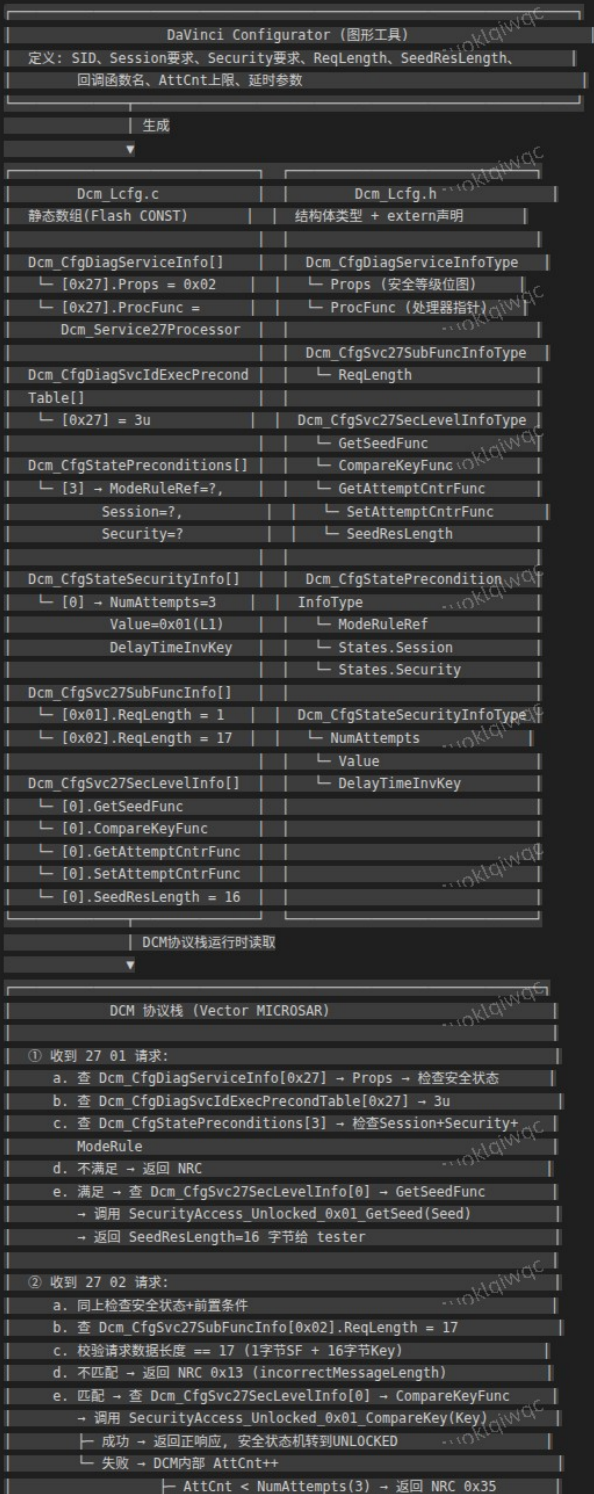
```
- SF 0x01 ReqLength: 1u (不变)
- SF 0x02 ReqLength: 5u → key=4字节
+ SF 0x02 ReqLength: 17u → key=16字节

- SeedResLength: 4u → seed=4字节
+ SeedResLength: 16u → seed=16字节
```

第六层: OEM回调实现 — [SecurityAccess.c](#)

```
// 使用 SecurityAccess.h 中的 OEM_SEED_LEN / OEM_KEY_LEN 宏
// 决定数组大小和循环边界
```

三、完整调用链路图



```
└─ AttCnt >= NumAttempts → 返回 NRC 0x36
+ 启动 DelayTimeInvKey=1000ms 延时
→ 调用 SetAttemptCounterFunc 写入NVM
|
| 回调
▼
SecurityAccess.c (OEM回调实现, dsar_fw 版本)
SecurityAccess_Unlocked_0x01_GetSeed() 行143:
① 检查 g_PoweronLINRC37Flag → TRUE则返回 0x37
② 检查 g_securitylevel1[1] (Static Seed_flag)
FALSE → SecurityAccess_GetSeed() 生成 OEM_SEED_LEN 字节随机数
存到 g_securitylevel1[2..1], 设置Static_Seed_flag
车型判断:
P301/P201/P567/C206_10/E001_10/E202_10 → SM4分支
复制到 sP301L1Seed[16]
else → 位运算分支, 拼成 uint32 sSeedL1
TRUE → 复用 g_securitylevel1[2..1] 的上次种子
③ 返回 Seed (长度从 SeedResLength=16 控制)
SecurityAccess_Unlocked_0x01_CompareKey() 行229:
① 清除 g_securitylevel1[1] (Static Seed_flag = FALSE)
写 Nvm 持久化
② 车型判断生成期望key:
P301/P201 → P301_L1Cipher 做 SM4加密(sP301L1Seed)
P567 → P567_L1Cipher 做 SM4加密
E001_10/E202_10 → E001_10_L1Cipher 做 SM4加密
C206_10 → C206_10_L1Cipher 做 SM4加密
else → SecurityAccess_GetKey(sSeedL1) 位运算
③ SecurityAccess_CompareKey(期望Key, 收到的Key, OEM_KEY_LEN)
匹配 → DCM_E_POSITIVERESPONSE, 日志[Normal]
不匹配 → g_securitylevel1[0]>=MAX_ATT CNT-1 ?
DCM_E_EXCEEDNUMBEROFATTEMPTS : DCM_E_INVALIDKEY
日志[Moderate]
SecurityAccess_Unlocked_0x01_GetAttemptCounter() 行327:
返回 g_securitylevel1[0] 给 DCM
SecurityAccess_Unlocked_0x01_SetAttemptCounter() 行353:
写入 g_securitylevel1[0], 并写 Nvm 持久化
```

四、g_securitylevel1[20] NVM 数组布局

Byte 0: AttCnt (尝试计数器, NVM持久化)
Byte 1: Static_Seed_flag (是否复用种子)
Byte 2..17: Seed 备份 (最多16字节)

五、0x19 / 0x85 变更项在调用链中的位置

0x19 和 0x85 共用上述第一层到第四层机制, 区别在第五层用自己的 SubFuncInfo。

变更点一: Dcm_CfgDiagServiceInfo[] 的 Props 字段

服务	旧值	新值	含义变化
0x19 (ReadDTC)	0x01u	0x00u	从「需要L1解锁」→「无需安全解锁」
0x85 (ControlDTC)	未变	未变	—
0x27 (SecurityAccess)	0x03u	0x02u	从「需L3级别安全状态」→「需L2级别」

Props 是位图: bit0 对应 L1 unlocked, bit1 对应 L2 unlocked, DCM 处理每个服务请求时, 第一步就检查当前安全状态是否覆盖 Props 要求的所有 bit。

变更点二: Dcm_CfgDiagSvcIdExecPrecondTable[]

服务	旧值	新值	指向的 Dcm_CfgStatePreconditions 条目
0x19	0u	2u	从 DefaultModeRule → 更严格的 mode rule
0x27	0u	3u	从 DefaultModeRule → 更严格的 mode rule
0x85	0u	3u	从 DefaultModeRule → 更严格的 mode rule

这个表是 DCM 服务执行的第二道检查 (先过 Props 的安全等级检查, 再过 ExecPrecond)。索引从 0u 变为 2u / 3u, 意味着 P301 的 DaVinci 中新增了 mode rule 条目, 对这 3 个服务绑定了更严格的执行条件。

六、变更影响总结

变更项	影响
SF 0x02 ReqLength 5 → 17	收到 key 长度不匹配的请求会被 DCM 协议栈直接拒绝(NRC 0x13)
SeedResLength 4 → 16	返回给 tester 的 seed 变为 16 字节
Props 0x03 → 0x02 (27服务)	降低了 27 服务本身的准入安全等级门禁
ExecCondRef 0 → 3 (27/19/85)	3个服务增加了 mode rule 前置条件检查
ExecCondRef 0 → 2 (19服务)	19服务要求更严格的状态机条件
Props 0x01 → 0x00 (19服务)	19服务不再需要安全解锁即可使用

变更的本质是: DaVinci 全量生成的配置文件替换, 两个工程之间在 mode rule 数量、安全等级要求、DID 列表上都有差异, 所以不仅是 27 服务的 seed/key 长度变了, 所有相关的服务配置都跟随 P301 工程的设定一起变了。